

Assignment 1 - 12415912 - Mehmed Mulalic

1. Overview

This document describes the design decisions, data structures, and the workflow of creating an Abstract Syntax Tree (AST). The project consists of a scanner file in Flex and a parser file in Bison in combination with an intermediate C file to create a symbol table, connect the scanner and parser, and to generate the AST using DOT representation. Finally, the project includes a Makefile which builds the project and creates the AST image with Graphviz.

2. Project Structure

The project consists of the following files:

File	Description
Makefile	Build files and generate the AST image as well as file cleanup.
intermediate.h	Header file, including struct definitions and function declarations.
intermediate.c	AST generation and symbol table implementation.
parser.y	Bison parser file. Dictates grammar rules and semantic actions.
scanner.l	Flex scanner file. Provides input file tokenization.
test_function.c	A simple C test function to showcase AST generation.

3. AST Data Structure

The Abstract Syntax Tree is built from a single node type defined as a C struct named `ASTNode`. This struct contains a label variable in addition to two `ASTNode` child structs which allow the creation of binary operations. This setup limits the tree generation by allowing only the creation of binary operations since there are only two `ASTNode` child nodes without any extra additions.

4. Symbol Table

The symbol table stores information about declared variables within a function. It is implemented as a fixed-size (of size 512) global array of `SymbolStatement` entries. The `SymbolStatement` is a struct defined to have a name and a type where the name represents the identifier and the type is the variable type. This struct, in combination with the insert and lookup method, provides a means to populate the table during declaration parsing and consult during statement parsing.

The insert method is called only in the `direct_declarator` grammar rule in the parser when a variable declaration is detected. The current type is stored in the global `current_type` variable set by the `type_specifier` grammar rule. The lookup method is called only in the `primary_expression` grammar rule when an identifier is used in a statement.

5. Parser and Scanner Integration

The Bison parser was adjusted such that it contains two blocks of code at the beginning to include the intermediate header file as well as to define data types to be assigned by grammar rules. By including the `%union` keyword, grammar rules can return either either a `char*` value (for identifiers and constants) or a `ASTNode*` value (for expressions and statements) by including a `<strval>` or `<node>` data type, respectively, to tokens or types. Later on, grammar rules assigns identifiers, constants, statement lists, and creates nodes for expressions and statements with the `$$` keyword.

The assignment is possible due to the scanner which stores tokens in `strval` for identifiers and constants through the addition of `strdup(yytext)` inside `check_type()`. By storing tokens, the parser can access the variable through `$1`, `$2`, etc. which are used during binary operations.

```
%code requires {
    #include "intermediate.h"
}

%union {
    char *strval;
    ASTNode *node;
}
```

6. DOT Visualization

After parser calls the `yyparse()` method, the AST is stored in the `root_node` global variable pointer. Afterwards, the `print_dot()` method runs where it creates a DOT file by recursively traversing the tree and writing instructions for Graphviz by including a unique ID assigned to each node. The output file is named `test_function.dot` and is generated automatically when `cparser test_function.c` is executed. The Makefile includes processing the DOT file to create an image in PNG format.

7. Design Decisions

The `ASTNode` contains only two node children which limits the tree to contain only binary operations, thus excluding the possibility of having some segments with additional children nodes.

The intermediate file contains a global `current_type` variable to store the variable type defined by the parser. This variable is later used when creating a `SymbolStatement` without additional parameters, reducing complexity.

The scanner stores all constants as strings. Since the assignment requires only AST visualization, there is no need for any type conversion. Additionally, for unary operations, such as increment (`++`) and decrement (`--`), the DOT output can not represent the ordering of the operation, i.e., whether the operation is a suffix or a prefix.

Node initialization is created with `calloc` rather than `malloc` to ensure that all pointer fields are initialized to NULL to mitigate segmentation faults during `print_dot_node` during tree traversal.